

UE5 C++ MASTERY ÇALIŞMA MERKEZİ · SUJET

05

SHOOTER SAM

Animasyonlu, ateş eden bir üçüncü şahıs oyuncu ile onu Behavior Tree ve NavMesh kullanarak avlayan bir yapay zeka düşman kurmak.

SÜRE

7 saat 34 dakika

DERS SAYISI

50 ders

ALİŞTİRMA

9 zorunlu + 1 bonus

XP

175 (+40 bonus)

BÖLÜM 0 — GENEL BİLGİLER VE KURALLAR

0.1 Önsöz

Bu, kursun son ve en ileri düzey bölümü. Buraya kadar öğrendiğin her şey — composition, delegate, trace, timer — burada animasyon ve yapay zeka ile birleşiyor. Behavior Tree, ilk bakışta karmaşık görünebilir; ama aslında sadece bir if/else yığınının görsel, yeniden kullanılabilir bir organizasyonudur.

0.2 Amaç

Blend Space ve State Machine ile hızla duyarlı bir animasyon sistemi, hitscan tabanlı bir ateşleme mekaniği ve NavMesh + Behavior Tree + Blackboard kullanan, oyuncuyu gördüğünde kovalayıp ateş eden, görmediğinde son bilinen konuma giden bir AI düşman kurmak.

0.3 Genel Yönlendirmeler

- Blend Space'i kurmadan önce hangi animasyonun hangi hız aralığına karşılık geldiğini kâğıda çiz.
- Behavior Tree'yi büyütmeden önce en basit haliyle (tek Sequence) çalıştığını doğrula.
- Custom C++ Task yazmadan önce aynı davranışı hazır node'larla taklit etmeyi dene.
- Health Component'i Battle Blaster'dan olduğu gibi taşı — yeniden yazma.

0.4 Değerlendirme Kriterleri (Kendi Kendine Savunma)

Bir 42 öğrencisinin projesini savunması gibi, aşağıdaki sorulara videoya veya nota bakmadan, kendi cümleleriyle cevap verebilmelisin:

- Blend Space'in girdi değişkeni neyi temsil ediyor ve bu değeri C++'tan nasıl besliyorsun?
- Sequence ile Selector düğümü arasındaki mantıksal fark nedir (VE/VEYA benzetmesiyle anlat)?
- Hitscan mermi ile fiziksel (spawn edilen) mermi arasındaki fark nedir, ne zaman hangisini seçersin?
- AI'nın “son bilinen oyuncu konumu” mantığı, oyunu nasıl daha gerçekçi hale getiriyor?

0.5 Zorunlu ve Yasak

✓ ZORUNLU

- Karakter hızını bir Blend Space'e bağlayarak animasyonu gameplay verisine göre sürmek
- Ateşlemeyi hitscan line trace ile yapmak
- AI kararlarını Behavior Tree + Blackboard üzerinden yönetmek (Tick içine gömülü if/else yerine)

✗ YASAK

- Animasyon geçişlerini elle (state machine kullanmadan) bool bayrak yığınlarıyla yönetmek
- AI hareket/karar mantığını doğrudan AI Controller'ın Tick'ine yazmak
- Health Component'i Battle Blaster'dakinden farklı, tekrar sıfırdan yazmak

0.6 XP ve Rank Sistemi

Bu bölümün her alıştırması bir XP değeri taşır. Zorunlu kısmı eksiksiz bitirince **175 XP** kazanırsın; bonus dahil üst sınır **215 XP**'dir (bonus, ancak zorunlu %100 tamamlandıktan sonra

açılır). Kazandığın XP'yi Notion'daki “XP Takip Sistemi” veritabanında işaretleyerek genel kurs rankını görebilirsin — bu bölümü bonusuyla bitirince **UE5 Mastery — Efsane** rankına yaklaşırsın.

BÖLÜM I — ZORUNLU KISIM

Alıştırmalar birbiri üzerine inşa olacak şekilde sıralanmıştır — bir öncekini bitirmeden sonrakine geçme. Her alıştırma, ilgili Notion sayfasındaki “Konu Anlatımı” bölümünün karşılık gelen kısmına dayanır.

EXERCISE 00 · 15 XP

Third Person Kurulumu ve Girdi

İlgili dersler: Third Person Template · Vector2D Input Value · Mouse Input

Amaç — Hazır Third Person Template'i inceleyip iki eksenli (Vector2D) hareket girdisini kurmak.

► Yapılacaklar

- Third Person Template projesini incele; karakter hareket kodunun nerede olduğunu bul.
- WASD hareketini bir Vector2D Input Action'a bağla.
- Fare girdisiyle kamera döndürmeyi ekle.

⚠ Kısıtlar

- Şablonun tüm mantığını silip sıfırdan yazma — önce nasıl çalıştığını anla, üzerine ekle.

✓ Beklenen Sonuç

- Karakter 8 yönde hareket edebiliyor, kamera fareyle serbestçe dönüyor.

EXERCISE 01 · 20 XP

Blend Space

İlgili dersler: Skeletal Animations 101 · The Animation Blueprint · Blend Spaces · Connecting Animation To Gameplay

Amaç — Karakterin hızını bir Blend Space'e bağlayarak yürüme/koşma animasyonları arasında yumuşak geçiş sağlamak.

► Yapılacaklar

- Bir Blend Space asset'i oluştur, hız eksenine örnek animasyonlar yerleştir.
- Animation Blueprint'te GetVelocity().Size() değerini bir float değişkenine yaz.
- Bu değişkeni Blend Space'in girdisine bağla.

⚠ Kısıtlar

- Animasyon geçişini Blend Space yerine elle if/else ile animasyon değiştirerek yapma.

✓ Beklenen Sonuç

- Karakter hızlandıkça yürümeden koşmaya oransal, sarsıntısız bir geçiş oluyor.

EXERCISE 02 · 15 XP

Durum Makinesi

İlgili dersler: Animation State Machines · Is Player Airborne? · Additive Animations

Amaç — Karakterin Yerde/Havada durumları arasında geçiş yapan bir Animation State Machine kurmak.

► Yapılacaklar

- Animation Blueprint'te bir State Machine oluştur: Grounded ve Airborne durumları.
- Havada olup olmama koşuluna göre geçiş kur.
- İsteğe bağlı: yukarı/aşağı nişan alma için bir Additive Animation katmanı ekle.

⚠ Kısıtlar

- Geçiş koşulunu her frame yeniden hesaplarken pahalı bir işlem yapma — basit bir bool kontrolü yeterli.

✓ Beklenen Sonuç

- Karakter zıpladığında Havada, yere değince otomatik Yerde durumuna geçiyor.

EXERCISE 03 · 15 XP

Silah Aktörü

İlgili dersler: Gun Actor · Spawning The Gun · Attaching To Socket

Amaç — Karakterden bağımsız bir Gun Actor sınıfı yazıp onu bir iskelet socket'ine bağlamak.

► Yapılacaklar

- Ayır bir Gun Actor sınıfı oluştur (composition — karakterin bir parçası değil, sahip olduğu bir nesne).
- Oyun başında spawn edip karakterin el socket'ine attach et.
- Namlu ucu için bir Scene Component (Muzzle) ekle.

⚠ Kısıtlar

- Silahı karakter sınıfının içine gömülü bir mesh olarak yazma — ayrı Actor olmalı.

✓ Beklenen Sonuç

- Silah, karakterin eline doğru konumda ve rotasyonda bağlı görünüyor.

EXERCISE 04 · 20 XP

Hitscan Ateşleme

İlgili dersler: Shooting Input · Player View Point · Bullet Line Trace · Gun Particle Effects · Bullet Damage

Amaç — Ateş edildiğinde oyuncunun bakış noktasından bir hitscan line trace göndererek anlık hasar vermek.

► Yapılacaklar

- Ateşleme eylemini Enhanced Input'a bağla.
- Oyuncunun bakış noktasından (Player View Point) bir line trace başlat.
- Vurulan noktaya bir parçacık efekti ekle, vurulan Actor'e ApplyDamage ile hasar ver.

⚠ Kısıtlar

- Fiziksel bir mermi Actor'ü spawn etme — bu bölümde hitscan tercih edilmeli.

✓ Beklenen Sonuç

- Ateş edildiğinde hedefe anında (gecikmesiz) hasar uygulanıyor.

EXERCISE 05 · 15 XP

Can Sistemi ve Ölüm Animasyonu

İlgili dersler: Character Health · Death Animation

Amaç — Battle Blaster'daki Health Component'i karaktere uygulayıp can sıfırlandığında ölüm animasyonunu oynatmak.

► Yapılacaklar

- Health Component'i karakter Actor'üne CreateDefaultSubobject ile ekle.
- Can sıfırlandığında Health Component'in delegate'ini dinleyip bir “Öldü” durumuna geç.
- Ayı bir Yaşam/Ölüm state machine'i ile ölüm animasyonunu tetikle.

⚠ Kısıtlar

- Health Component'i sıfırdan yeniden yazma — önceki bölümden aynen taşı.

✓ Beklenen Sonuç

- Can sıfırlanınca karakter ölüm animasyonunu oynatıp kontrol kaybediyor.

EXERCISE 06 · 20 XP

AI Controller ve Hareket

İlgili dersler: AI Controller & Focus · AI Movement & NavMesh · AI Line Of Sight

Amaç — Bir AI Controller yazıp NavMesh üzerinde oyuncuya doğru hareket eden ve görüş hattını kontrol eden bir düşman kurmak.

► Yapılacaklar

- Bir AAIController alt sınıfı yaz, seviyeye bir NavMeshBoundsVolume ekle.
- AI'ın Focus'unu oyuncuya ayarla.
- AI ile oyuncu arasında engelsiz görüş hattı olup olmadığını bir trace ile kontrol et.

⚠ Kısıtlar

- NavMeshBoundsVolume'u eklemeyi unutma — AI'lar yürünebilir alan olmadan hareket edemez.

✓ Beklenen Sonuç

- Düşman, NavMesh üzerinde oyuncuya doğru yol bulup hareket edebiliyor.

EXERCISE 07 · 30 XP

Behavior Tree ve Blackboard

İlgili dersler: BehaviorTrees & Blackboards · Tasks and Sequences · Selector Node · Decorators · Services · Last Known Player Location

Amaç — Sequence/Selector/Decorator/Service kullanarak “oyuncuyu görüyorsa yaklaş ve ateş et, görmüyorsa son bilinen konuma git” mantığını kurmak.

► Yapılacaklar

- Bir Blackboard oluştur, “OyuncuKonumu” ve “GörüşVar” gibi key'ler ekle.
- Bir Service ile görüş hattı kontrolünü düzenli aralıklarla Blackboard'a yaz.
- Bir Selector altında iki Sequence kur: (görüyorsa → yaklaş+ateş et) / (görmüyorsa → son konuma git).

⚠ Kısıtlar

- Tüm mantığı tek bir dev Sequence içine sıkıştırma — Selector ile dallandır.

✓ Beklenen Sonuç

- AI, oyuncu görüş alanından çıktığında son gördüğü yere gidip orada araştırıyor.

EXERCISE 08 · 25 XP

Özel C++ Task

İlgili dersler: Custom C++ Task · Shoot Task

Amaç — Hazır node'ların karşılamadığı bir davranış için kendi C++ BTTaskNode'unu yazmak.

► Yapılacaklar

- UBTTaskNode'dan miras alan bir sınıf oluştur, ExecuteTask'ı override et.
- OwnerComp.GetAIOwner()->GetPawn() ile kontrol edilen Pawn'a eriş.
- Pawn'ın ateş etme fonksiyonunu çağırıp görevi tamamla (FinishExecute).

⚠ Kısıtlar

- FinishExecute çağırmayı unutma — aksi halde Behavior Tree o dalda takılı kalır.

✓ Beklenen Sonuç

- AI, davranış ağacı bu task'a ulaştığında gerçekten ateş ediyor.

BÖLÜM II — BONUS (İSTEĞE BAĞLI)

Bonus, zorunlu kısım tam ve eksiksiz çalışır durumda olmadan başlanmamalıdır — tıpkı 42'deki mantık gibi: önce sağlam temel, sonra süsleme.

BONUS · +40 XP (zorunlu %100 tamamlanınca açılır)

Can Barı HUD'u ve Ses

İlgili dersler: Health Bar HUD · Sound Cues

Amaç — Oyuncu ve/veya düşman için ekranda bir can barı göstermek ve vuruş/ölüm anlarına ses eklemek.

► Yapılacaklar

- UMG'de bir can barı widget'ı tasarla, Health Component'in delegate'ine bağla.
- Bir Sound Cue oluşturup vuruş anında rastgele varyasyonla çal.
- Ölüm anında ayrı bir ses tetikle.

⚠ Kısıtlar

- Can barını her Tick'te sorgulamak yerine delegate tabanlı güncelle.

✓ Beklenen Sonuç

- Can barı gerçek zamanlı güncelleniyor, vuruş/ölüm anlarında ses duyuluyor.

TESLİM VE ÖZ-DEĞERLENDİRME

Bu bölümü ve dolayısıyla kursun tamamını bitirdiğinde, sıfırdan bir Actor yazmaktan davranış ağacı tasarlayan bir yapay zekaya kadar uzanan tam bir yolculuğu tamamlamış olacaksın. Bu sujet'teki her “Beklenen Sonuç”u tek tek kendi projende doğrulayabiliyorsan, kursu videoya ihtiyaç duymadan tekrar inşa edebilecek seviyedesin demektir.